Energy-Efficient and Fast Memristor-based Serial Multipliers Applicable in Image Processing

Seyed Erfan Fatemieh[†], Bahareh Bagheralmoosavi[†], and Mohammad Reza Reshadinezhad[†,*]

Abstract

Memristive Processing In-Memory (PIM) is one of the promising techniques for overcoming the Von-Neumann bottleneck. Reduction of data transfer between processor and memory and data processing by memristors in data-intensive applications reduces energy consumption and processing time. Multipliers are one of the fundamental arithmetic circuits that play a significant role in data-intensive processing applications. The computational complexity of multipliers has turned them into one of the arithmetic circuits affecting PIM's efficiency and energy consumption, for example, in convolution operations. Serial material implication (IMPLY) logic design is one of the methods of implementing arithmetic circuits by applying emerging memristive technology that enables PIM in the structure of crossbar arrays. The authors propose unsigned and signed array multipliers using serial IMPLY logic in this paper. The proposed multipliers have improved significantly compared to State-Of-the Art (SOA) by applying the proposed Partial Product Units (PPUs) and overlapping computational steps. The number of computational steps, energy consumption, and required memristors of the proposed 8-bit unsigned array multiplier are improved by up to 36%, 31%, and 47% compared to the classic designs. The proposed 8-bit signed multiplier has also improved the computational steps, energy consumption, and required memristors by up to 59%, 54%, and 45%. The performance of the proposed multipliers in the applications of Gaussian blur and edge detection is also investigated, and the simulation results have shown an improvement of 31% in energy consumption and 33% in the number of computational steps in these applications.

Keywords

Memristor, IMPLY Logic, Multiplier, Image Processing, In-Memory Computing, Processing In-Memory

1 Introduction

In the era of expanding data-intensive applications, industry and researchers face challenges in designing efficient processing systems. Image processing, the Internet of Things (IoT), and machine learning are among the most critical data-intensive applications [1]. For a long time, the Von-Neumann architecture has been the conventional architecture for these applications, which separates the processing units and the memory (data storage) at a distance. Data movement between processing units and memory in today's processing systems is associated with several challenges, such as reducing efficiency and increasing energy consumption, known as "Von-Neumann's bottleneck" [1-6]. Since the 80s, the performance of processors has doubled every two years, while the performance of memory has doubled every 10 years [7, 8]. However, due to the limitations of the power wall and transistors' scalability, processor performance growth has slowed in recent years, with performance doubling almost every 20 years [8-10]. Increased leakage currents lead to higher power consumption and reduced efficiency growth [11-13]. Also, the energy consumption of data movement between memory and the processing unit is challenging [4]. Various solutions have been proposed to improve energy consumption and overcome the Von-Neumann bottleneck, including near-memory processing (placing several cache levels close to the processor or 3D stacking of Dynamic Random Access Memory (DRAM) on the processor) and PIM [1, 2, 5, 6]. In the PIM paradigm, the memory array can process data and perform logical

[†] S. E. Fatemieh (erfanfatemieh@eng.ui.ac.ir), B. Bagheralmoosavi (b.mousavi@eng.ui.ac.ir), and M. R. Reshadinezhad (m.reshadinezhad@eng.ui.ac.ir) are with the Department of Computer Architecture, Faculty of Computer Engineering, University of Isfahan, Isfahan 8174673441, Iran.

[*] Corresponding author: Mohammad Reza Reshadinezhad.

and arithmetic operations besides storing data. Conventional memories, including Static RAM (SRAM) and emerging nonlinear nanotechnologies such as memristors (Resistive RAM (RRAM)), are being applied in academia and industry for integration with arithmetic and logic units [2, 14].

RRAM is known for its high retention time in different conditions, nanometric dimensions, compatibility with the Complementary Metal Oxide Semiconductor (CMOS) manufacturing process, high switching speed, and low energy consumption [1-5, 15]. Memristors have been commercialized by Intel, Fujitsu, Panasonic, etc., and researchers are looking to replace them with DRAM and flash memory cells [3, 15]. The resistance of the RRAM cell changes to Low Resistance State (LRS) and High Resistance State (HRS) when voltage/current is applied in the direction of polarity or vice versa, and otherwise, its resistance value remains constant [1-4]. Several methods, including IMPLY, Fast and Energy-Efficient Logic (FELIX), and Memristor Aided LoGIC (MAGIC), have been proposed to implement and execute logical and arithmetic instructions using memristors as emerging memory cells [16-18]. IMPLY is a stateful logic design method that represents logic zero and one by HRS and LRS, respectively [4, 5, 18]. Stateful logic methods can help overcome the Von-Neumann bottleneck when designing arithmetic circuits for PIM. This approach can reduce energy consumption by minimizing data transmission [5, 20]. With its simple and compact structure, IMPLY logic is well-suited for coordination with crossbar arrays.

Several applied logic gates and arithmetic operations in data-intensive applications, including adders and multipliers, use stateful logic methods such as IMPLY [1, 16, 20-25]. The structural complexity of multipliers is more than that of adder circuits. Computing the product of two numbers requires more time (number of computational cycles) and energy consumption than computing their sum. Considering the importance of multipliers as fundamental arithmetic circuits in applications such as convolution and multiply-accumulate (MAC) operations, reducing the hardware complexity and computational delay directly affects overall efficiency in processing these applications [11, 26]. Several multipliers have been introduced using the IMPLY logic design method based on parallel architecture to reduce the computational steps [24, 25]. However, this topology increases hardware complexity and there is no proper coordination between parallel structure and crossbar arrays [1, 2].

Signed and unsigned multiplications are among the main arithmetic operations in data processing applications. Designing stateful memristive multipliers compatible with the structure of crossbar arrays can be a priority for PIM. Several algorithms and methods have been introduced to implement energy/area efficient fast unsigned and signed multipliers regardless of the implementation technology. Dadda, array, and add & shift multipliers are commonly used for unsigned multiplication [27]. Additionally, signed multiplication is often carried out using the Baugh-Wooley multiplier with the Dadda reduction tree, radix-2 Booth multiplier, and array multiplier [27]. In this paper, the authors' main goal is to present IMPLY-based serial unsigned and signed multipliers compatible with crossbar arrays to reduce the computational steps and energy consumption compared to SOA. The main contributions of this paper are:

1. Implementation of the classic add & shift and classic Dadda multipliers by applying the IMPLY logic method in serial topology, calculating the number of memristors and computational steps, and estimating the energy consumption of the n-bit structure,
2. Implementation of add & shift, radix-2 Booth, Baugh-Wooley (with Dadda reduction tree) as classic signed multipliers using IMPLY logic in serial topology and calculating circuit analysis metrics of the n-bit structures,
3. Implementation of classic unsigned and signed array multipliers with Carry Save Adder (CSA) tree based on the IMPLY method in serial architecture and calculating the number of computational steps, the number of memristors, and the energy consumption of the n-bit structure,
4. Proposing nine serial IMPLY-based PPUs to be applied in unsigned and signed array multipliers based on the CSA tree to reduce the number of computational steps and the energy consumption of the n-bit serial multiplier,
5. Calculation of the number of computational steps, memristors, and the energy consumption of the proposed improved n-bit unsigned and signed array multipliers and
6. Investigating the functionality of the proposed multipliers and SOA in Gaussian blur and edge detection applications and evaluating the circuit analysis metrics improvement.

The rest of the article is divided into four sections. The second section describes IMPLY as a stateful logic method, common implementation architectures based on it, and the reasons for selecting serial topology. Then, the basic cells required to implement IMPLY-based classic multipliers are introduced in subsection 2.2. Serial multipliers introduced based on the IMPLY design method in recent years are assessed in the last subsection of Section 2, along with the implementation details of classic unsigned and signed multipliers. Classic unsigned and signed array multipliers, improved PPUs, and proposed array multipliers are reported in the third section. The fourth section presents circuit-level simulation results, their analysis and evaluation, and a comparison of the proposed circuit with previous works. In the last subsection of section 4, the results of application-level simulation (Gaussian blur and edge detection) are reported, along with the evaluation and comparison. Finally, in the fifth section, the article concludes.

2 Previous Works

2.1 IMPLY logic and its implementation architectures

IMPLY logic operation ($p \rightarrow q \equiv \neg p + q$) can be implemented by two memristors, p and q, connected to the ground through a resistor (R_G), as shown in Figure 1(a). By simultaneously applying voltages V_{COND} and V_{SET} to memristors p and q, respectively, the result of $p \rightarrow q$ is stored in memristor q. The binary value stored in each memristor can be changed to the logic zero value (HRS) by applying the V_{RESET} voltage to the memristor. The truth table of $p \rightarrow q$ is written in Table 1. Two conditions must be met for the IMPLY gate, based on two memristors p and q, to function properly: 1) $V_{COND} < V_{SET}$, and 2) $R_{ON} \ll R_G \ll R_{OFF}$ [5]. Various logic gates, adders, and multipliers are implemented based on IMPLY logic [4, 5, 16, 20-23]. Four different architectures can be applied to implement different IMPLY-based circuits. In the serial architecture, the memristors (input, output, and work) are all placed in a row or column, and only one IMPLY operation is executed in each computational cycle [21]. The structure of serial architecture is shown in Figure 1(b). In parallel architecture, the input, output, and work memristors are placed in different rows/columns and are connected to the R_G resistor of each row/column by CMOS switches [21]. Also, rows/columns can be connected through CMOS switches. Assume no dependency between memristors in different rows/columns in each cycle. In that case, one IMPLY operation is executed in each row/column based on the step order of the implementation algorithm [21, 23]. Semi-serial and semi-parallel architectures have also been proposed to trade-off processing time and hardware complexity between serial and parallel architectures [21, 22].

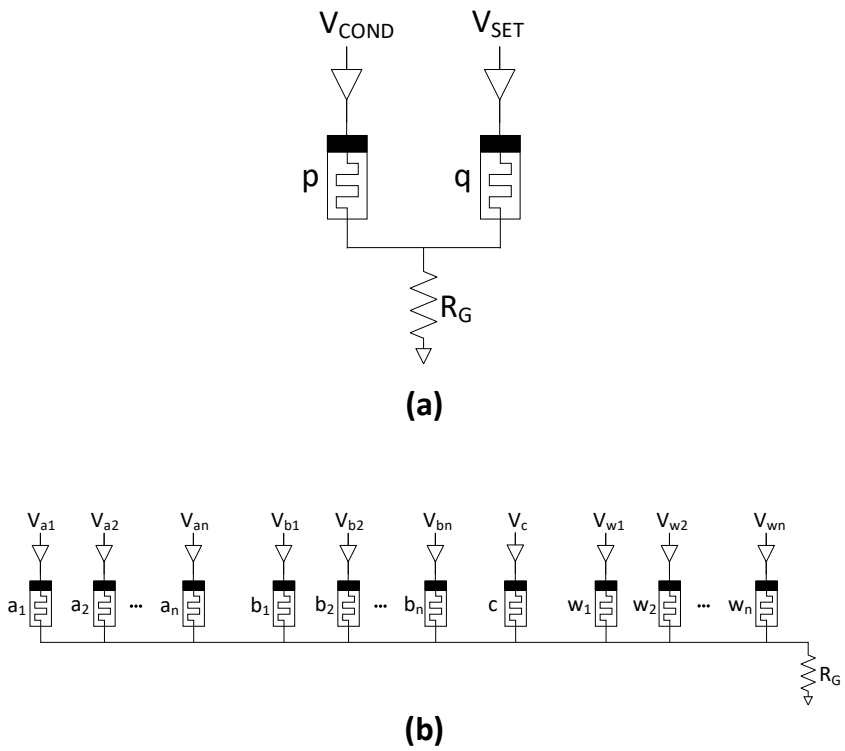

Figure 1: (a) Memristive IMPLY gate, and (b) Memristive IMPLY-based serial architecture [28].